Medical HMS — Frontend Architecture Guide (Next.js)

> **Audience:** Junior developers joining the Medical HMS frontend team. **Stack:** Next.js (App Router) · TypeScript (strict) · SCSS Modules + CSS variables · TanStack Query · Zustand · react-hook-form + Zod **Read this fully before writing any code.** It tells you *what to do*, *what NOT to do*, *where every file goes*, and *why.*

---

1. The 10-second summary

1. **Reusable, dumb UI** lives in `src/core/` (our "core components" — CustomForm, CustomTable, etc.).
2. **Each page owns everything it needs** — its API calls, its logic hook, its view, its styles — **inside its own page folder**.
3. Every component is split into **4 files**: view (`.tsx`), logic (`use*.ts`), styles (`.module.scss`), types (`.types.ts`).
4. **Never put logic or API calls directly in the view file.** The view only displays.
5. **Never** redesign the JSON/schema-driven CustomForm/CustomTable — pass config, don't hardcode UI.

If you remember nothing else, remember those five.

---

2. Folder structure

```
medical-hms/
├── src/
│   ├── app/                              # ROUTING + PAGES. Each page folder is self-contained.
│   │   ├── (auth)/
│   │   │   └── login/page.tsx
│   │   │
│   │   ├── (doctor)/                      # ← "doctor module" (route group)
│   │   │   ├── layout.tsx                 #   doctor shell: sidemenu + auth guard
│   │   │   └── appointments/              # ← ONE PAGE = ONE SELF-CONTAINED FOLDER
│   │   │       ├── page.tsx               #   VIEW (thin, dumb)
│   │   │       ├── useAppointments.ts     #   LOGIC hook (state + handlers)
│   │   │       ├── appointments.api.ts    #   API CALLS for this page  ← co-located
│   │   │       ├── appointments.types.ts  #   types for this page
│   │   │       ├── appointments.module.scss
│   │   │       └── components/            #   components used ONLY by this page
│   │   │           └── AppointmentCard/
│   │   │               ├── AppointmentCard.tsx
│   │   │               ├── AppointmentCard.module.scss
│   │   │               └── index.ts
│   │   │
│   │   ├── (admin)/                       # ← "admin module"
│   │   ├── (patient)/                     # ← "patient module"
│   │   ├── layout.tsx                     # root layout: providers, theme, fonts
│   │   └── globals.scss
│   │
│   ├── core/                              # ← REUSABLE dumb UI (our "CoreModule")
│   │   ├── custom-form/
│   │   │   ├── CustomForm.tsx
│   │   │   ├── useCustomForm.ts
│   │   │   ├── CustomForm.module.scss
│   │   │   ├── CustomForm.types.ts
│   │   │   └── index.ts
│   │   ├── custom-table/
│   │   ├── custom-modal/
│   │   ├── sidemenu/  chip/  info-tile/  stat-cards/ ...
│   │   └── index.ts                       # barrel: export * from each component
│   │
│   ├── lib/                               # shared infra (used by many pages)
│   │   ├── api/
│   │   │   └── client.ts                  # fetch wrapper + JWT injection (the ONLY place fetch is configured)
│   │   ├── auth/                          # session helpers, role guards
│   │   ├── query/                         # TanStack Query setup
│   │   └── utils/                         # generic helpers (date format, etc.)
│   │
│   ├── stores/                            # Zustand global client state (ui, filters)
│   ├── styles/                            # _tokens.scss, _mixins.scss, theme.scss
│   └── types/                             # truly global shared types
├── public/
├── next.config.ts
└── tsconfig.json                          # path aliases: @/core, @/lib, @/stores
```

The golden boundary

Folder	Contains	Allowed to call APIs?	Reusable?
`src/core/`	Dumb presentational UI	❌ **No** — receives data via props only	✅ Everywhere
`app/.../<page>/`	A page + its own logic + its own API	✅ **Yes** — in its `*.api.ts`	❌ Page-specific
`src/lib/`	Shared infra (api client, auth, utils)	✅ Setup only	✅ Everywhere

> **Rule:** `core/` components never know where data comes from. Pages fetch data and hand it to `core/` components as props.

3. The 4-file component pattern (DO THIS EVERY TIME)

Every non-trivial component is **four files**. This is how we keep your Angular-style `ts / html / scss` separation in React.

```
custom-form/
├── CustomForm.tsx          ← VIEW   (your .html) — JSX only, thin
├── useCustomForm.ts        ← LOGIC  (your .ts)   — state, handlers, effects. NO JSX.
├── CustomForm.module.scss  ← STYLES (your .scss) — scoped automatically
├── CustomForm.types.ts     ← TYPES  — interfaces, the JSON schema shape
└── index.ts                ← barrel — re-exports the component
```

View file (`.tsx`) — the "HTML". Stays dumb.

```
// CustomForm.tsx
'use client';
import styles from './CustomForm.module.scss';
import { useCustomForm } from './useCustomForm';
import { FormField } from './components/FormField';
import type { CustomFormProps } from './CustomForm.types';

export function CustomForm({ schema, onSubmit }: CustomFormProps) {
  // ALL logic comes from the hook. The view never computes anything heavy.
  const { values, errors, handleChange, handleSubmit } = useCustomForm(schema, onSubmit);

  return (
    <form className={styles.form} onSubmit={handleSubmit}>
      {schema.map((field) => (
        <FormField
          key={field.keyName}
          field={field}
          value={values[field.keyName]}
          error={errors[field.keyName]}
          onChange={handleChange}
        />
      ))}
      <button type="submit" className={styles.submit}>Save</button>
    </form>
  );
}
```

Logic file (`use*.ts`) — the "TS". Zero JSX.

```ts
// useCustomForm.ts
import { useState } from 'react';
import type { FormEvent } from 'react';
import type { InputField } from './CustomForm.types';

export function useCustomForm(schema: InputField[], onSubmit: (v: Record<string, unknown>) => void) {
  const [values, setValues] = useState(() => buildInitialValues(schema));
  const [errors, setErrors] = useState<Record<string, string>>({});

  const handleChange = (key: string, value: unknown) => {
    setValues((prev) => ({ ...prev, [key]: value }));
  };

  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    const validationErrors = validate(schema, values);
    if (Object.keys(validationErrors).length) {
      setErrors(validationErrors);
      return;
    }
    onSubmit(values);
  };

  return { values, errors, handleChange, handleSubmit };
}

function buildInitialValues(schema: InputField[]) {
  return Object.fromEntries(schema.map((f) => [f.keyName, f.value ?? '']));
}
function validate(schema: InputField[], values: Record<string, unknown>) {
  const errors: Record<string, string> = {};
  for (const f of schema) {
    if (f.isRequired && !values[f.keyName]) errors[f.keyName] = `${f.inputName} is required`;
  }
  return errors;
}
```

Types file (`.types.ts`) — the JSON schema shape

```ts
// CustomForm.types.ts
export interface InputField {
  inputName: string;
  inputType: 'text' | 'number' | 'select' | 'date' | 'file' | 'richtext';
  keyName: string;
  value?: unknown;
  isRequired?: boolean;
  data?: { label: string; value: string }[];   // options for select
  conditionalDisplay?: { dependsOn: string; showWhen: unknown[] };
}

export interface CustomFormProps {
  schema: InputField[];
  onSubmit: (values: Record<string, unknown>) => void;
}
```

Styles file (`.module.scss`) — scoped

```
// CustomForm.module.scss
.form {
  display: flex;
  flex-direction: column;
  gap: var(--space-md);
}
.submit {
  background: var(--color-primary);   // always use theme tokens, never raw hex
  border-radius: var(--radius);
  color: #fff;
}
```

> **When is 4 files overkill?** For a truly tiny component (a `Chip`, an `InfoTile` with no logic) you may keep the logic inline in the `.tsx` and skip the hook. Use your judgment: **if it has state, effects, or API calls → split out the hook.** When in doubt, split.

---

4. User-type "modules" = Route Groups

Each user type gets a **route group** — a folder in parentheses. The parentheses mean "group these routes and give them a shared layout, but don't add to the URL."

```
app/
├── (admin)/layout.tsx       → wraps every admin page (sidemenu, admin guard)
├── (doctor)/layout.tsx      → wraps every doctor page
└── (patient)/layout.tsx     → wraps every patient page
```

```
// app/(doctor)/layout.tsx — the "doctor module shell"
import { Sidemenu } from '@/core';
import { requireRole } from '@/lib/auth';

export default async function DoctorLayout({ children }: { children: React.ReactNode }) {
  await requireRole('doctor');        // guard — redirects if not a doctor
  return (
    <div className="app-shell">
      <Sidemenu role="doctor" />
      <main>{children}</main>
    </div>
  );
}
```

> This is the direct equivalent of your Angular per-user-type feature modules + route guards.

---

5. Co-located API calls (our chosen pattern)

Each page folder contains its own `*.api.ts`. API functions for a page live next to the page, not in a global services folder. They all go through the shared `lib/api/client.ts` (which injects the JWT and base URL — never re-configure fetch yourself).

```
// app/(doctor)/appointments/appointments.api.ts
import { api } from '@/lib/api/client';
import type { Appointment } from './appointments.types';

export const appointmentsApi = {
  list: (doctorId: string) =>
    api.get<Appointment[]>(`/doctors/${doctorId}/appointments`),

  cancel: (id: string) =>
    api.patch(`/appointments/${id}/cancel`),
};
```

Then the page's logic hook calls it through **TanStack Query** (handles loading, caching, refetch — replaces manual subscribe/unsubscribe):

```
// app/(doctor)/appointments/useAppointments.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { appointmentsApi } from './appointments.api';

export function useAppointments(doctorId: string) {
  const qc = useQueryClient();

  const { data: appointments = [], isLoading } = useQuery({
    queryKey: ['appointments', doctorId],
    queryFn: () => appointmentsApi.list(doctorId),
  });

  const cancel = useMutation({
    mutationFn: appointmentsApi.cancel,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['appointments', doctorId] }),
  });

  return { appointments, isLoading, cancelAppointment: cancel.mutate };
}
```

```
// app/(doctor)/appointments/page.tsx — VIEW. Thin. Composes core components.
'use client';
import { CustomTable } from '@/core';
import { useAppointments } from './useAppointments';
import { appointmentColumns } from './appointments.types';

export default function AppointmentsPage() {
  const { appointments, isLoading, cancelAppointment } = useAppointments(currentDoctorId);

  if (isLoading) return <p>Loading…</p>;

  return (
    <CustomTable
      tableData={{ columns: appointmentColumns, dataSource: appointments }}
      rowActions={[{ type: 'cancel', onClick: (row) => cancelAppointment(row.id) }]}
    />
  );
}
```

> **Notice the layering:** `page.tsx` (view) → `useAppointments.ts` (logic) → `appointments.api.ts` (API) → `lib/api/client.ts` (transport). Each layer has one job.

---

6. State management

	Use	Lives in
Data from the NestJS backend (server state)	**`@tanstack/react-query`**	page's `use*.ts` hook
To call the API	a `*.api.ts` function → `lib/api/client.ts`	page folder / `lib`
Global client-only state (theme, open modals, active filters shared across pages)	**`zustand`** store	`src/stores/`
Local single-component state	`useState` (built into React)	the hook

The two packages, and the line between them:

> **`@tanstack/react-query`** = **SERVER state** (anything that lives in the database — patients, appointments, bills). **`zustand`** = **CLIENT state** (UI-only things the backend never knows about — sidebar open/closed, active theme, selected filters). If it comes from an API → React Query. If it's pure UI → Zustand. Never store API data in Zustand.

Do not build a global Redux store for server data. TanStack Query *is* your server-state cache.

6.1 Install

```
npm install @tanstack/react-query zustand
```

6.2 One-time setup: wrap the app in the Query provider

React Query needs a provider at the root. Create it once:

```
// src/lib/query/QueryProvider.tsx
'use client';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useState } from 'react';

export function QueryProvider({ children }: { children: React.ReactNode }) {
  // one client per browser session
  const [client] = useState(() => new QueryClient({
    defaultOptions: {
      queries: { staleTime: 60_000, retry: 1 },  // tune as needed
    },
  }));
  return <QueryClientProvider client={client}>{children}</QueryClientProvider>;
}
```

```
// src/app/layout.tsx — wrap everything once
import { QueryProvider } from '@/lib/query/QueryProvider';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body><QueryProvider>{children}</QueryProvider></body>
    </html>
  );
}
```

6.3 Using React Query (server state) — in the page's hook

You already saw this in §5. `useQuery` for reads, `useMutation` for writes, `invalidateQueries` to refetch after a change:

```
// inside app/(doctor)/appointments/useAppointments.ts
const { data, isLoading, error } = useQuery({
  queryKey: ['appointments', doctorId],          // cache key — unique per data set
  queryFn: () => appointmentsApi.list(doctorId), // the actual fetch
});

const cancel = useMutation({
  mutationFn: appointmentsApi.cancel,
  onSuccess: () => qc.invalidateQueries({ queryKey: ['appointments', doctorId] }),
});
```

That gives you loading, error, caching, dedupe, and background refetch for free — the work you used to do by hand with RxJS `subscribe` / `unsubscribe`.

> **`queryKey` rule:** include every variable the data depends on (e.g. `['appointments', doctorId]`). Same key = shared cache; different key = separate cache entry.

6.4 Using Zustand (client state) — a global store

A Zustand store is the closest thing to your Angular `@Injectable()` singleton. Define it once in `src/stores/`, use it from any component — no provider needed.

```
// src/stores/ui.store.ts
import { create } from 'zustand';

interface UiState {
  isSidebarOpen: boolean;
  theme: 'default' | 'hospital-b';
  toggleSidebar: () => void;
  setTheme: (theme: UiState['theme']) => void;
}

export const useUiStore = create<UiState>((set) => ({
  isSidebarOpen: true,
  theme: 'default',
  toggleSidebar: () => set((s) => ({ isSidebarOpen: !s.isSidebarOpen })),
  setTheme: (theme) => set({ theme }),
}));
```

```
// use it anywhere — just call the hook, select only what you need
'use client';
import { useUiStore } from '@/stores/ui.store';

export function SidebarToggle() {
  const isOpen = useUiStore((s) => s.isSidebarOpen);   // select a slice → re-renders only when it changes
  const toggle = useUiStore((s) => s.toggleSidebar);
  return <button onClick={toggle}>{isOpen ? 'Close' : 'Open'} menu</button>;
}
```

> **Zustand tip:** always select a *slice* (`useUiStore((s) => s.theme)`), not the whole store, so components re-render only when the value they use changes.

---

7. Styling & theming

- **SCSS is our styling language.** Install `sass`. Two layers:
  1. `Component.module.scss` — scoped per component (use for everything component-specific).
  2. `src/styles/` — global theme tokens + mixins.
- **Theme via CSS custom properties** (runtime variables), so we can support per-hospital white-label themes later by swapping one attribute.

```
// src/styles/_tokens.scss
:root {
  --color-primary: #0e7c7b;
  --color-danger:  #d64545;
  --space-md:      16px;
  --radius:        8px;
}
[data-theme='hospital-b'] { --color-primary: #1a56db; }  // white-label override
```

- **In components, always use tokens:** `color: var(--color-primary)` — **never** raw hex like `#0e7c7b` inside a component.

---

8. Naming conventions

Thing	Convention	Example
Component file	`PascalCase.tsx`	`CustomTable.tsx`
Hook file	`useCamelCase.ts`	`useAppointments.ts`
API file	`kebab-or-camel.api.ts`	`appointments.api.ts`
Types file	`name.types.ts`	`appointments.types.ts`

Thing	Convention	Example
Styles	`Name.module.scss`	`CustomTable.module.scss`
Route folder	`kebab-case`	`appointment-history/`
Boolean vars	`is/has/can` prefix	`isLoading`, `hasAccess`

---

9. ✅ DO

- ✅ Split every real component into **view / hook / styles / types**.
- ✅ Keep `page.tsx` **thin** — it reads from a hook and renders. No business logic.
- ✅ Put a page's **API calls in its own `*.api.ts`** inside the page folder.
- ✅ Route all HTTP through **`lib/api/client.ts`**.
- ✅ Use **TanStack Query** for backend data; **Zustand** for global UI state.
- ✅ Pass **JSON config** to `CustomForm` / `CustomTable` — extend the schema, never hardcode fields.
- ✅ Use **theme tokens** (`var(--color-…)`) for all colors and spacing.
- ✅ Reuse `core/` components; add new shared ones to `core/` + export from `core/index.ts`.
- ✅ Keep **strict TypeScript** — type every prop and API response.

10. ❌ DON'T

- ❌ Don't put API calls or business logic **inside a `.tsx` view file**. → Move to the hook / `*.api.ts`.
- ❌ Don't make `core/` components fetch data. They receive props only.
- ❌ Don't use `any` to silence TypeScript. Fix the type.
- ❌ Don't write raw `fetch()` in pages. Use `lib/api/client.ts`.
- ❌ Don't recreate Angular `NgModule`s or look for dependency injection — they don't exist here.
- ❌ Don't try to make `.tsx` a pure HTML template. The hook holds the logic instead.
- ❌ Don't put raw hex colors / magic numbers in component styles. Use tokens.
- ❌ Don't duplicate a component — if two pages need it, promote it to `core/`.
- ❌ Don't hardcode form fields or table columns — drive them from the JSON schema.
- ❌ Don't share one page's `*.api.ts` with another page. If shared, move it to `lib/`.

---

11. Checklist before you open a PR

- [ ] View file has **no** business logic or API calls.
- [ ] Logic is in a `use*.ts` hook; API is in `*.api.ts`.
- [ ] All HTTP goes through `lib/api/client.ts`.
- [ ] Types defined — no `any`.
- [ ] Colors/spacing use theme tokens, not raw values.
- [ ] Reused a `core/` component where one existed (didn't duplicate).
- [ ] Files named per §8.
- [ ] Component renders loading + error states (TanStack Query gives you these).

---

12. Glossary

Term	Meaning
Route group	`(name)` folder — groups pages under a shared layout without changing the URL
Barrel file	`index.ts` that re-exports many modules so you import from one path
Hook	A `use*` function holding React state/logic — our replacement for component `.ts` logic
Schema-driven	UI built by passing a JSON config (our `CustomForm` / `CustomTable` pattern)
Co-located	Files kept next to where they're used (a page's api/logic/styles in its own folder)
Token	A CSS variable for a themeable value (`--color-primary`)
Dumb / Presentational	A component that only displays props, never fetches data